

Lab 15 - 1st Derivative Filtering

June 7, 2025

1 Theory

The Prewitt and Sobel operators are first derivative edge detection filters used in image processing to highlight regions with high spatial derivatives, typically corresponding to edges.

1.1 Prewitt Operator

The Prewitt operator uses two 3×3 convolution kernels to approximate the gradient of the image intensity function:

Horizontal:

$$\begin{bmatrix} -1 & -1 & -1 \\ 0 & 0 & 0 \\ 1 & 1 & 1 \end{bmatrix}$$

Vertical:

$$\begin{bmatrix} -1 & 0 & 1 \\ -1 & 0 & 1 \\ -1 & 0 & 1 \end{bmatrix}$$

1.2 Sobel Operator

The Sobel operator is similar to the Prewitt operator but gives more weight to the center pixels, making it more sensitive to edges:

Horizontal:

$$\begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{bmatrix}$$

Vertical:

$$\begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix}$$

Both operators are applied to grayscale images to detect horizontal and vertical edges by convolving the kernels with the image.

```
[1]: import numpy as np
from PIL import Image
import matplotlib.pyplot as plt
```

```
[2]: def Prewitt_Operator_horizontal(input_image):
    img = input_image.resize((400,400), Image.Resampling.LANCZOS)

    # convert to numpy array
    numpy_image = np.array(img)
    # array for padding
    array_b = np.zeros((402,402))

    # to pad initial array with zeros in all side
    array_b[1:401,1:401] = numpy_image

    #defining filter
    filter_array = np.array([[ -1,-1,-1],
                             [ 0,0,0],
                             [ 1,1,1]])

    #creating empty list
    lst = []

    for i in range(400):
        for j in range(400):
            #extracting part of array equal to filter size
            array_c = array_b[i:(3+i),j:(3+j)]

            #applying filter
            array_mul = np.multiply(filter_array,array_c)
            array_sum = np.sum(array_mul)

            # putting calculated value in list
            lst.append(array_sum)

    # resizing lst to shape of original array
    final_array = np.resize(lst,(400,400))

    final_image = Image.fromarray(final_array)
    final_image= final_image.convert("L")

    return final_image
```

```
[3]: def Prewitt_Operator_vertical(input_image):
    img = input_image.resize((400,400), Image.Resampling.LANCZOS)
```

```

# convert to numpy array
numpy_image = np.array(img)
# array for padding
array_b = np.zeros((402,402))

# to pad initial array with zeros in all side
array_b[1:401,1:401] = numpy_image

#defining filter
filter_array = np.array([[ -1,0,1],
                        [ -1,0,1],
                        [ -1,0,1]])

#creating empty list
lst = []

for i in range(400):
    for j in range(400):
        #extracting part of array equal to filter size
        array_c = array_b[i:(3+i),j:(3+j)]

        #applying filter
        array_mul = np.multiply(filter_array,array_c)
        array_sum = np.sum(array_mul)

        # putting calculated value in list
        lst.append(array_sum)

# resizing lst to shape of original array
final_array = np.resize(lst,(400,400))

final_image = Image.fromarray(final_array)
final_image= final_image.convert("L")

return final_image

```

```

[4]: def Sobel_Operator_horizontal(input_image):
    img = input_image.resize((400,400), Image.Resampling.LANCZOS)

    # convert to numpy array
    numpy_image = np.array(img)
    # array for padding
    array_b = np.zeros((402,402))

```

```

# to pad initial array with zeros in all side
array_b[1:401,1:401] = numpy_image

#defining filter
filter_array = np.array([[ -1,-2,-1],
                        [ 0,0,0],
                        [ 1,2,1]])

#creating empty list
lst = []

for i in range(400):
    for j in range(400):
        #extracting part of array equal to filter size
        array_c = array_b[i:(3+i),j:(3+j)]

        #applying filter
        array_mul = np.multiply(filter_array,array_c)
        array_sum = np.sum(array_mul)

        # putting calculated value in list
        lst.append(array_sum)

# resizing lst to shape of original array
final_array = np.resize(lst,(400,400))

final_image = Image.fromarray(final_array)
final_image= final_image.convert("L")

return final_image

```

```

[5]: def Sobel_Operator_vertical(input_image):
    img = input_image.resize((400,400), Image.Resampling.LANCZOS)

    # convert to numpy array
    numpy_image = np.array(img)
    # array for padding
    array_b = np.zeros((402,402))

    # to pad initial array with zeros in all side
    array_b[1:401,1:401] = numpy_image

    #defining filter
    filter_array = np.array([[ -1,0,1],

```

```

        [-2,0,2],
        [-1,0,1]])

#creating empty list
lst = []

for i in range(400):
    for j in range(400):
        #extracting part of array equal to filter size
        array_c = array_b[i:(3+i),j:(3+j)]

        #applying filter
        array_mul = np.multiply(filter_array,array_c)
        array_sum = np.sum(array_mul)

        # putting calculated value in list
        lst.append(array_sum)

# resizing lst to shape of original array
final_array = np.resize(lst,(400,400))

final_image = Image.fromarray(final_array)
final_image= final_image.convert("L")

return final_image

```

```

[6]: # reading image and converting to gray scale
img = Image.open('img/prakash.jpg').convert('L')
# Calling function
Prewitt_Operator_horizontal_image = Prewitt_Operator_horizontal(img)
Prewitt_Operator_vertical_image = Prewitt_Operator_vertical(img)
Sobel_Operator_horizontal_image = Sobel_Operator_horizontal(img)
Sobel_Operator_vertical_image = Sobel_Operator_vertical(img)

```

```

[7]: fig = plt.figure()
fig.set_figheight(20)
fig.set_figwidth(20)

#plotting original image
fig.add_subplot(2,3,1)
plt.imshow(img, cmap='gray')
plt.title('original')

#plotting filtered image
fig.add_subplot(2,3,2)
plt.imshow(Prewitt_Operator_horizontal_image, cmap='gray')

```

```
plt.title('Prewitt_Operator_horizontal')

#plotting filtered image
fig.add_subplot(2,3,3)
plt.imshow(Prewitt_Operator_vertical_image, cmap='gray')
plt.title('Prewitt_Operator_Vertical')

#plotting filtered image
fig.add_subplot(2,3,4)
plt.imshow(Sobel_Operator_horizontal_image, cmap='gray')
plt.title('Sobel_Operator_horizontal')

#plotting filtered image
fig.add_subplot(2,3,5)
plt.imshow(Sobel_Operator_vertical_image, cmap='gray')
plt.title('Sobel_Operator_vertical')
```

[7]: Text(0.5, 1.0, 'Sobel_Operator_vertical')



```

[8]: # reading image and converting to gray scale
img = Image.open('img/tiger.png').convert('L')
# Calling function
Prewitt_Operator_horizontal_image = Prewitt_Operator_horizontal(img)
Prewitt_Operator_vertical_image = Prewitt_Operator_vertical(img)
Sobel_Operator_horizontal_image = Sobel_Operator_horizontal(img)
Sobel_Operator_vertical_image = Sobel_Operator_vertical(img)

fig = plt.figure()
fig.set_figheight(20)
fig.set_figwidth(20)

#plotting original image
fig.add_subplot(2,3,1)
plt.imshow(img, cmap='gray')
plt.title('original')

#plotting filtered image
fig.add_subplot(2,3,2)
plt.imshow(Prewitt_Operator_horizontal_image, cmap='gray')
plt.title('Prewitt_Operator_horizontal')

#plotting filtered image
fig.add_subplot(2,3,3)
plt.imshow(Prewitt_Operator_vertical_image, cmap='gray')
plt.title('Prewitt_Operator_Vertical')

#plotting filtered image
fig.add_subplot(2,3,4)
plt.imshow(Sobel_Operator_horizontal_image, cmap='gray')
plt.title('Sobel_Operator_horizontal')

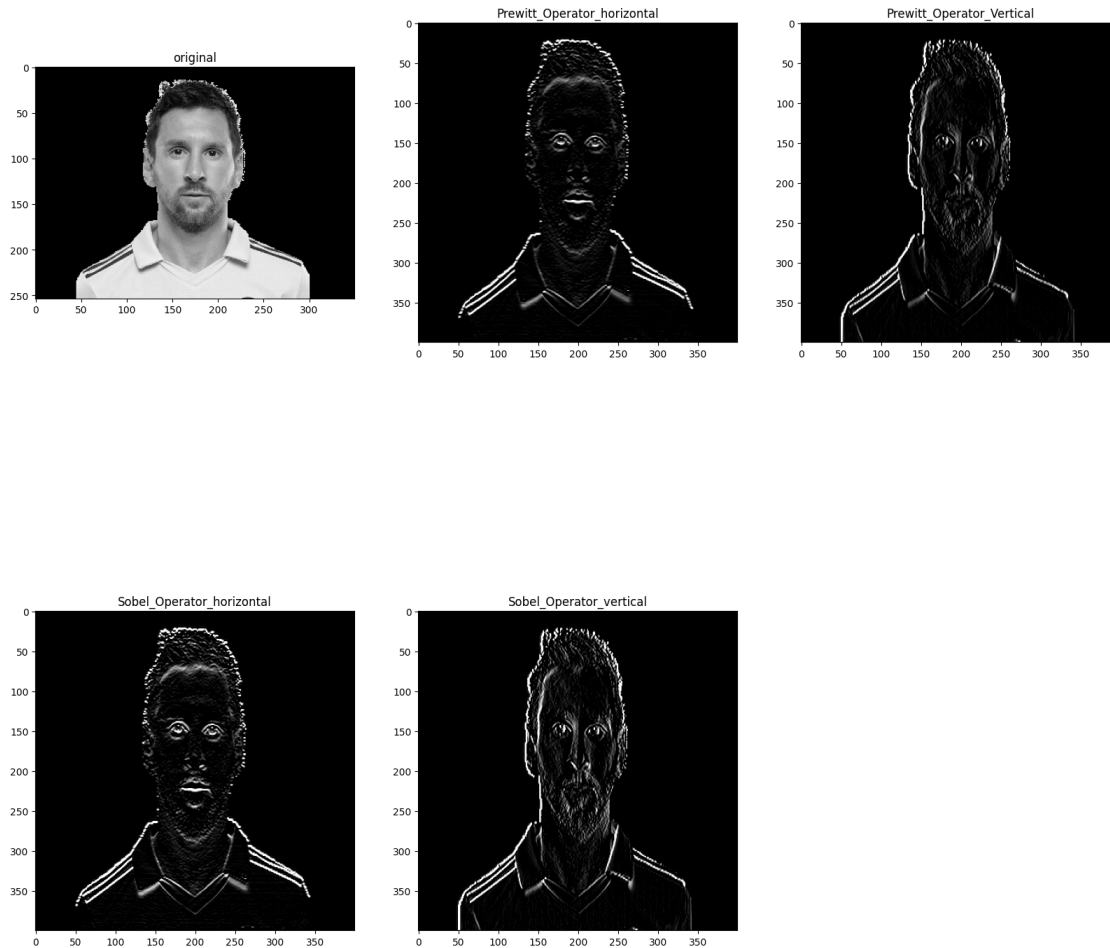
#plotting filtered image
fig.add_subplot(2,3,5)
plt.imshow(Sobel_Operator_vertical_image, cmap='gray')
plt.title('Sobel_Operator_vertical')

```

```

[8]: Text(0.5, 1.0, 'Sobel_Operator_vertical')

```



2 Conclusion

The Prewitt and Sobel operators are effective for detecting edges in digital images. The Sobel operator, due to its higher emphasis on central pixels, often produces smoother and more pronounced edges compared to the Prewitt operator. These filters are fundamental in image analysis tasks such as feature extraction and object detection.